Vegetable Price Prediction using Linear Regression

A Mini Project Report

Submitted by: Hrishiti Mahajan (2427030719) & Saahir Nagpal (2427030687)
Course: B.Tech CSE Core | **Subject:** Predictive Analytics | **Section:** F
Submitted to: Dr. Lav Upadhyay

Abstract

In this project we tried to predict the daily price of vegetables in Indian mandis using machine learning. The dataset is in the same format as the government's Agmarknet records, but we want to be clear that it is simulated and not real. It has five vegetables (Tomato, Onion, Potato, Brinjal and Cauliflower) and five markets (Pune, Azadpur in Delhi, Bangalore, Lucknow and Kolkata) from January 2022 to December 2025, which gives 35,075 rows.

Our main model is Linear Regression. It uses past prices, rolling averages, market arrivals and the time of year to predict the next day's modal price. We compared it with a naive baseline (tomorrow's price = today's price) and with Ridge Regression. Linear Regression had an average error of about 25 rupees per quintal, around 1% of the price, and it was about 3% better than the baseline in MAE and RMSE. Since the data is synthetic, this project mainly shows that our pipeline works from start to end. It does not tell us how real mandi prices behave.

1. Introduction

Anyone who buys vegetables knows how quickly prices change. One week tomatoes are cheap and the next week they cost double. This happens because of the season, rainfall, festivals, transport problems and how much produce reaches the market on a given day. For farmers and traders it is a bigger problem than an annoyance. A farmer who sends a truck of onions to the mandi on the wrong day can lose a lot of money, and even a rough idea of where prices are going can help people decide when to sell, store or wait.

We picked this topic because the government already publishes daily mandi prices, so it felt like a real problem with real data behind it. It was also a good way to practise the full machine learning process on time series data, which is quite different from the classification examples we usually do in class.

Problem statement: Given the recent price history, market arrivals and the time of year for a vegetable in a particular mandi, predict tomorrow's modal price (in rupees per quintal) for that vegetable in that mandi.

Objectives:

- Clean and prepare mandi price data for machine learning.
- Build features from past prices, arrivals and season without leaking any future information.
- Train Linear Regression (and Ridge for comparison) for next-day price prediction.
- Compare the models with a simple naive baseline using proper error metrics, and be honest about what the results mean.

Scope: We only predict one day ahead, and each vegetable in each market is treated as its own series (5 × 5 = 25 series). We did not use outside factors like weather or fuel prices.

2. Basic Concepts

Mandi, Agmarknet and modal price. A mandi is a wholesale market where farmers and traders buy and sell farm produce. Agmarknet is the government system that collects daily prices and arrivals from many mandis, and a lot of it is also on data.gov.in. For each commodity, market and date there are three prices: minimum, maximum and modal. The modal price is the price at which most of the trading happened that day, so we think it represents the market better than the average of min and max. That is why we chose it as our target. Prices are in rupees per quintal (1 quintal = 100 kg).

Time series. A time series is data recorded in order over time, like daily prices. The order matters because today's price depends on yesterday's (this is called autocorrelation), which is why guessing "tomorrow will be like today" works so well. Time series can also have seasonality (patterns that repeat every year) and trend (a slow rise or fall).

Linear Regression and Ridge. Linear Regression predicts a number as a weighted sum of the inputs, prediction = b0 + w1×feature1 + ... + wn×featuren, and finds the weights that make the squared errors as small as possible. We chose it because we can look at the weights and explain what each input does. Ridge Regression is the same thing with a penalty on large weights, controlled by alpha. It can help when the inputs are closely related, like our lag prices and rolling averages, so we tried alpha = 10.

Naive baseline. A baseline is a very simple method that a real model has to beat. For daily prices it is to predict that tomorrow's price equals today's. It sounds too simple, but for slow-moving prices it is hard to beat, so it is a fair test.

3. Dataset

We could not download real mandi data in bulk. The data.gov.in API mostly gives current or recent records, and getting several years of history would have taken more time than we had. So we used a synthetic dataset made to look like an Agmarknet export. Because of this, our results show that the method works, but not how real markets behave. The notebook can also read a real Agmarknet CSV, so only the file needs to be changed to run it on real data.

The synthetic data has a yearly seasonal pattern, a slow upward trend, random ups and downs, and a link between arrivals and price (more arrivals means lower prices). About 4% of the days were removed at random to copy the gaps in real reporting.

Property	Value
Rows / Period	35,075 rows, 1 Jan 2022 to 31 Dec 2025
Vegetables	Tomato, Onion, Potato, Brinjal, Cauliflower
Markets	Pune, Azadpur (Delhi), Bangalore, Lucknow, Kolkata
Columns	State, District, Market, Commodity, Variety, Arrival_Date, Min Price, Max Price, Modal Price, Arrivals_in_Tonnes

After cleaning, all 35,075 rows were kept (7,015 per vegetable). The summary numbers are below. The modal price has a standard deviation of about 672 rupees, so prices vary a lot across vegetables, markets and time. This is good because the model has something to learn from, and a typical error of 25 rupees is small compared to this spread.

Column	Mean	Minimum	Maximum
Min price (Rs/quintal)	2,014	942	5,407

Max price (Rs/quintal)	2,412	1,164	6,328
Modal price (Rs/quintal)	2,213	1,097	5,717
Arrivals (tonnes)	115.3	36.8	307.7

Exploratory data analysis

Before modelling we made three plots. Figure 1 (7-day rolling average of modal price) shows the level, trend and seasonal waves of every series. Figure 2 (box plot by month) checks if prices change with the time of year, and Figure 3 (arrivals vs modal price) checks if higher arrivals go with lower prices. Both suggest that seasonal features and arrivals are worth adding.

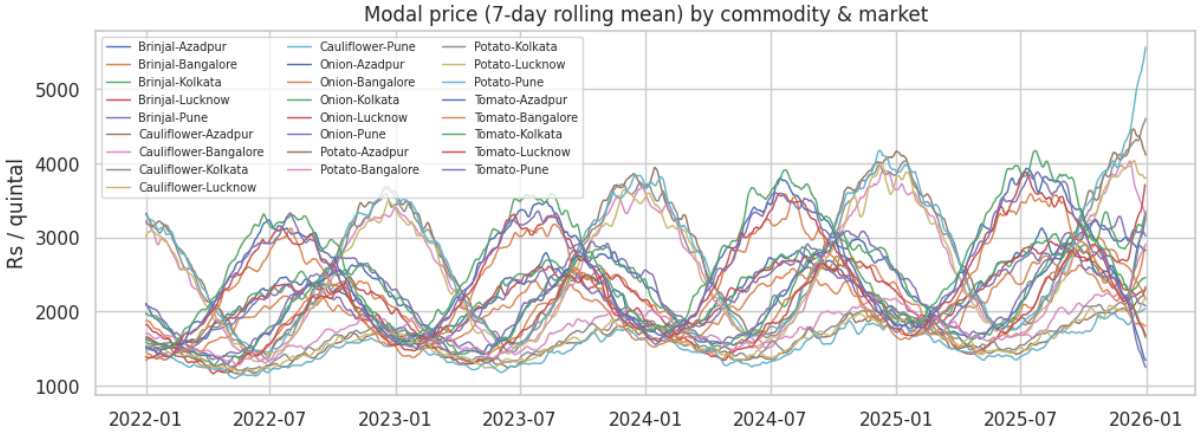
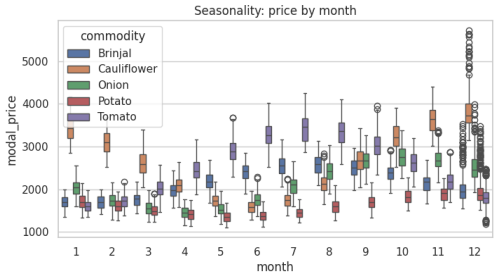


Figure 1: 7-day rolling mean of modal price by commodity and market



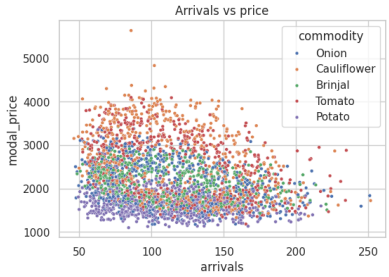


Figure 2 (left): Box plot of modal price by month. Figure 3 (right): Arrivals vs modal price

4. Tools Used

We did everything in Python on Google Colab, so nothing had to be installed on our laptops. We used pandas for loading, cleaning and making lag and rolling features, NumPy for number work and generating the synthetic data, scikit-learn for the models, scaling, one-hot encoding, pipelines and metrics, matplotlib and seaborn for plots, and joblib for saving the trained model.

5. Methodology

Our work has seven steps: load the data, clean it, explore it, make features, split by time, train the models and evaluate them.

5.1 Data cleaning

We wrote the cleaning step so that it should also work on a messy real file:

1. Made column names uniform (lowercase, underscores) and matched them loosely, so "Modal Price" and "Arrival_Date" become modal_price and date.
2. Converted dates to a proper format with dayfirst=True, because Indian dates are day first (01/02/2022 is 1 February).
3. Converted price and arrival columns to numbers (anything that failed was marked missing), then removed rows with a missing date or a modal price of zero or less.
4. Kept one row per commodity, market and date (the mean if there were duplicates), and dropped any series with fewer than 60 days of data.

Nothing was lost in our dataset because the synthetic data was already tidy, so the final shape stayed at 35,075 rows. These steps would matter much more with real data.

5.2 Feature engineering

This was the most important part. Our main rule was that every input must come from earlier days. We shifted everything by one day, so when the model predicts a day's price it has never seen that day's price, min, max or arrivals. If we skipped this the model would be cheating by looking at the answer, and the results would look great but mean nothing. This is called data leakage. The features for each series were:

Feature	What it means
lag_1, lag_7, lag_14	Modal price 1, 7 and 14 days ago
roll_mean_7, roll_std_7	Average and standard deviation (jumpiness) of the previous 7 days
roll_mean_30	Average modal price over the previous 30 days (longer trend)
spread_lag1	Yesterday's max price minus min price (rough volatility)
arrivals_lag1, arrivals_roll7	Yesterday's arrivals and the 7-day average arrivals (supply)
doy_sin, doy_cos	Sine and cosine of the day of the year (time of year)
commodity, market	Names converted to one-hot columns

We used sine and cosine for the time of year because the plain day number jumps from 365 back to 1 at New Year, even though 31 December and 1 January are one day apart. Sine and cosine make the year a smooth circle. Making these features needs up to 30 days of history, so the first 30 days of every series were dropped (35,075 to 34,325 rows).

5.3 Train and test split

We trained on the first 80% of the dates (31 Jan 2022 to 21 Mar 2025, 27,483 rows) and tested on the last 20% (22 Mar to 31 Dec 2025, 6,842 rows). We did not shuffle. A random split would put future days in the training set and the model could use them to predict the past, which never happens in real life. Testing only on later dates is a fair copy of real forecasting.

5.4 Pipeline and models

We used a scikit-learn Pipeline. Numeric features were scaled with StandardScaler and the commodity and market columns were one-hot encoded. The pipeline also means the scaler only learns from the training data, which avoids another small kind of leakage. We compared three approaches: the **naive baseline** (yesterday's price), **Linear Regression** with all features, and **Ridge Regression** (alpha = 10).

5.5 Evaluation metrics

- **MAE:** the average size of the error in rupees per quintal. An MAE of 25 means we are about 25 rupees off on average.
- **RMSE:** like MAE but it punishes big mistakes more, so a much larger RMSE means some large errors.
- **MAPE:** the average error as a percentage of the actual price, useful because 25 rupees matters more for a cheap vegetable.
- **R^2:** the share of the variation in prices that the model explains (1 is perfect).

6. Implementation Summary

The notebook follows the same order as the sections above, with each part in a separate cell. It sets a random seed of 42 so results can be repeated. A switch called DATA_SOURCE chooses between uploading a CSV, using the data.gov.in API or generating synthetic data, and for the final results we uploaded mandi_prices_dataset.csv (35,075 rows, 10 columns). A build_features function makes the lag, rolling, spread, arrivals and seasonal features using groupby and shift. After training and evaluation, the best model is saved with joblib as veg_price_linreg.joblib and used to predict for the latest day of every series.

7. Results and Discussion

7.1 Model comparison

The table shows how the models did on the test period, which they never saw during training.

Model	MAE (Rs)	RMSE (Rs)	MAPE (%)	R^2
Naive (yesterday's price)	26.019	34.465	1.035	0.998
Linear Regression	25.172	33.429	1.001	0.998
Ridge (alpha 10)	25.634	34.144	1.019	0.998

Linear Regression did best on MAE, RMSE and MAPE, but only slightly better than the baseline: MAE went down by about 3.3% (26.019 to 25.172) and RMSE by about 3.0% (34.465 to 33.429). Ridge came in between, so the extra regularisation did not help. We think this is because we have around 27,000 training rows and only about 11 numeric inputs, so overfitting is not a risk and shrinking the weights gives no benefit.

7.2 Why is R^2 the same for everything?

All three models have R^2 = 0.998. At first this looked too good to be true, but daily prices change very slowly, so even yesterday's price explains almost all of the variation in today's price. A high R^2 is therefore easy to get and does not tell us if the model is good. For us this was the biggest lesson: the better test is the comparison with the baseline, which shows what the model adds on top of what we get for free.

7.3 Is a 3% improvement meaningful?

Our honest answer is: a little, but not a lot. It shows up in all three metrics, so it is probably not random, and it suggests that rolling averages, arrivals and season add some real information. But in real life a farmer would not act differently because of an error of 25 rupees instead of 26. The gain is small because we predict only one day ahead, where the baseline is already very strong. We expect the features to help more for a longer horizon like one week, but we did not test this. With an average price of 2,213 rupees, an MAE of about 25 is around 1.1% of the price, so on a normal day the prediction is within about 1% of the real price.

7.4 Sample predictions

We also predicted the last day in the data (31 December 2025) with the saved model:

Commodity	Market	Actual price	Predicted price
Potato	Azadpur	2,418	2,439.6
Tomato	Kolkata	2,472	2,456.5
Onion	Azadpur	2,843	2,836.8
Cauliflower	Pune	5,717	5,643.7
Tomato	Lucknow	1,604	1,624.0

Most errors are a few tens of rupees. The largest is Cauliflower in Pune at about 73 rupees, which is still only around 1.3% of the price. A handful of predictions is not a proper test though, and the test-set metrics over 6,842 predictions are much more reliable.

7.5 Diagnostic plots

Figure 4 shows the actual vs predicted scatter (points should be near the diagonal), the histogram of residuals (should be centred around zero) and actual vs predicted over time for one series. Figure 5 shows the coefficients. Since the features were standardised, their sizes can be compared directly, and because Linear Regression gives one weight per feature it is easy to see which inputs matter. This is the main reason we picked it over a more complex model.

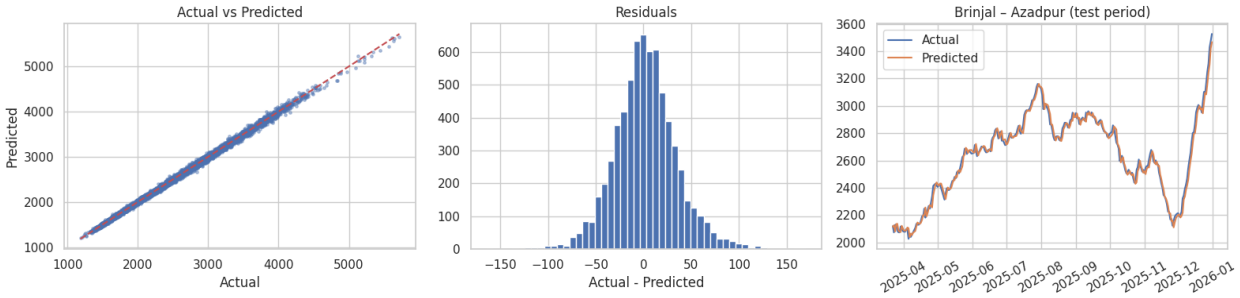


Figure 4: Actual vs predicted, residual histogram and time plot for the test period

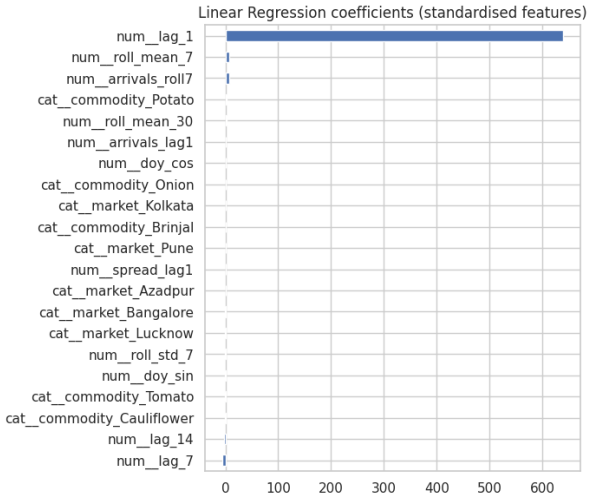


Figure 5: Linear Regression coefficients (standardised features)

8. Challenges We Faced

- **Getting real data.** The government portals are made for checking current prices, not for downloading years of history, so we had to use synthetic data. This is the biggest weakness of the project.
- **Avoiding data leakage.** Every feature had to use only past information. Using shift(1) before every rolling calculation was easy to forget.
- **Understanding the high R^2.** 0.998 felt suspicious, and working out why taught us to always compare with a baseline.
- **Gaps in the series.** With about 4% of days missing, a lag of 7 rows is not always exactly 7 calendar days. A more careful version would handle missing dates.
- **Time-based splitting.** We had to split by date instead of using the usual shuffled split.

9. Limitations

- The data is simulated, so real events like floods, strikes or policy changes are not included and we cannot say the model would work this well on real prices.
- It only predicts one day ahead, where the naive baseline is already very strong.
- Linear Regression cannot capture non-linear effects, like prices shooting up when supply drops below a certain level.
- We used a single train-test split and no time series cross-validation, so the results come from one test period.
- The model has few inputs (no weather, fuel prices, festivals or nearby markets), and its gain over the baseline is only about 3%.

10. Future Work

- Run the same notebook on real Agmarknet or data.gov.in data (the most important next step).
- Add inputs like rainfall, temperature, fuel prices, festival dates and prices from nearby markets.
- Try Random Forest and XGBoost, and use time series cross-validation (TimeSeriesSplit).
- Predict a week ahead instead of one day, and build lags from calendar dates to handle missing days.
- Make a simple web app where a user picks a vegetable and a market and sees the predicted price. The model is already saved with joblib.

11. Conclusion

We built a Linear Regression model that predicts next-day vegetable prices with about 1% error on average. We went through the whole process: cleaning, building features from past information only, splitting by time, comparing three models and checking them with four metrics. Linear Regression beat the naive baseline by about 3% in MAE and RMSE, so the extra features added a little information, while Ridge did not help.

The main things we learned are that a very high R^2 can be misleading for time series, that a simple baseline is the fairest test, and how easy it is to leak future information by mistake. Because we used synthetic data, this project mainly shows that our pipeline works, and we cannot claim it would forecast real mandi prices this well. The next step is to run it on real data and try longer forecast horizons.

References

1. Agmarknet, Government of India: agmarknet.gov.in (daily mandi prices and arrivals).
2. Open Government Data Platform India: data.gov.in, "Current Daily Price of Various Commodities from Various Markets (Mandis)".
3. scikit-learn documentation: scikit-learn.org (LinearRegression, Ridge, Pipeline, TimeSeriesSplit).
4. pandas documentation: pandas.pydata.org.
5. Google Colab: colab.research.google.com.